

Pathfinder

AI-Assisted Career Intelligence and
Skill Development Platform

Technical Documentation

Technology:	React, TypeScript, Express, LangChain
Frontend:	Vite, Zustand
Backend:	Express 5, Zod
Persistence:	Supabase
AI:	Multi-provider LLM architecture

Contents

1	Overview	3
2	Problem and Solution Design	3
2.1	Problem	3
2.2	Design Decision	3
3	System Architecture	4
3.1	High-Level Architecture	4
3.2	Technology Stack	4
3.3	Project Scale	4
4	Recommendation Engine	4
4.1	Skill State	4
4.2	Skill Gaps	5
4.3	Resource Scoring	5
4.4	Dependency Ordering	5
5	Bayesian Mastery Model	6
5.1	Prior	6
5.2	Likelihood	6
5.3	Bayesian Update	6
5.4	Decision Rule	6
6	Language Model Architecture	7
6.1	Structured Output	7
7	AI Safety and Guardrails	7
7.1	Input Protection	7
7.2	Output Protection	8
8	User Experience	8
8.1	Offline-First Interaction	8
8.2	Concurrent Requests	8
8.3	Onboarding	9
8.4	Discover	9
8.5	Connection State	9
9	API	9
10	Performance and Code Quality	9
11	Testing	10

12 Security	10
13 Deployment	11
13.1 Local Development	11
13.2 Production	11
14 Failure Handling	11
15 Innovation	12
15.1 Model as Renderer	12
15.2 Evidence-Based Explanations	12
15.3 Probabilistic Skipping	12
15.4 Honest Degradation	12
16 Limitations and Future Work	13
16.1 Current Limitations	13
16.2 Future Improvements	13
17 Conclusion	13

1 Overview

Pathfinder is an AI-assisted career intelligence platform that generates personalized learning paths from a learner’s current skills, interests, target career, and assessment evidence.

The system is designed around a central principle:

The model explains; the deterministic engine decides.

Career selection, skill-gap calculation, resource selection, dependency ordering, and mastery decisions are handled deterministically. Language models are used only where natural language is required.

The system supports offline operation and deterministic fallbacks, so model or network failures do not make the core product unavailable.

2 Problem and Solution Design

2.1 Problem

A learner does not simply need a list of courses. They need to know:

1. Where they are currently starting from.
2. What skills are actually missing.
3. Which resources should be completed first.
4. Why each resource is relevant.
5. Which topics can safely be skipped.

A conventional architecture that sends a career goal directly to an LLM can produce incorrect prerequisites, unverifiable recommendations, and explanations that disagree with the generated plan.

2.2 Design Decision

Pathfinder separates reasoning from language generation.

Profile → Deterministic Engine → Learning Plan → LLM Explanation

The model is used at five language-oriented edges:

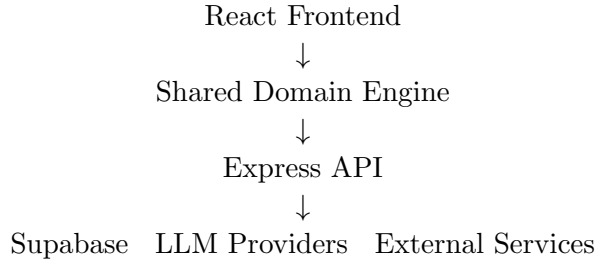
Operation	Purpose	Restriction
<code>extractGoal</code>	Map text to known goal	Cannot invent goals
<code>narrate</code>	Explain generated plan	Cannot change plan
<code>converse</code>	Generate response	Cannot change facts
<code>onboardingFollowup</code>	Ask constrained follow-up	Closed option set
<code>onboardingIntro</code>	Summarize answers	Cannot alter answers

Table 1: Language-model integration points

Every operation has a deterministic fallback.

3 System Architecture

3.1 High-Level Architecture



The domain layer is shared between frontend and backend and has no React dependency. This prevents separate implementations of the recommendation logic from diverging.

3.2 Technology Stack

Technology	Role
React 19	User interface
TypeScript	Static typing
Vite	Build system
Zustand	Client state management
Express 5	Backend API
Zod	Runtime validation
LangChain	Model abstraction
Supabase	Authentication and persistence

Table 2: Technology stack

3.3 Project Scale

Metric	Value
Source files	55
Lines of code	15,515
Domain layer	2,698 lines
API endpoints	25
Catalogue skills	27
Learning resources	35
Goal tracks	4
Assessment items	66
Assessed skills	25
Model providers	5
E2E checks	29

Table 3: Project scale

4 Recommendation Engine

4.1 Skill State

The learner state is represented by:

$$L \in \{0, \dots, 5\}^{27}$$

where each element represents the estimated level of one catalogue skill.

The state combines:

- Self-ratings.
- Completed resources.
- Experience floors.

Stronger evidence overrides weaker evidence.

4.2 Skill Gaps

For skill s :

$$gap_s = \max(0, target_s - current_s)$$

Career goals define target skill levels and weights.

4.3 Resource Scoring

Resources are ranked using marginal contribution to remaining skill gaps.

$$score(r) = \sum_s [\min(teaches(r, s), target_s) - current_s]^+ \cdot weight_s \times levelFit(r) + 0.6|tags(r) \cap interests| - \frac{hours(r)}{100}. \quad (1)$$

The level-fit factor is:

$$levelFit(r) = \begin{cases} 1.0 & \text{ready} \\ 0.7 & \text{slightly short} \\ 0.25 & \text{far off} \end{cases}$$

After each selection, the learner state is updated and the remaining gaps are recalculated.

The path is capped at 12 resources.

4.4 Dependency Ordering

Selected resources are emitted in dependency-respecting order.

The ordering process uses topological selection based on the learner's current skill state.

Ties are resolved by:

1. Difficulty.
2. Resource type.
3. Length.

This prevents advanced projects from appearing before the knowledge required to complete them.

5 Bayesian Mastery Model

Pathfinder does not treat a quiz result as absolute proof of mastery.

Instead, mastery is represented by a posterior distribution over six latent levels.

5.1 Prior

The prior follows geometric decay:

$$P(L = k) \propto \text{decay}^{|k - \text{assumed}|}$$

with:

$$\text{decay} = \begin{cases} 0.62 & \text{assumed evidence} \\ 0.35 & \text{verified evidence} \end{cases}$$

5.2 Likelihood

For a question with difficulty d :

$$P(\text{correct} \mid L, d) = \begin{cases} 0.85 & L \geq d \\ 0.28 & L < d \end{cases}$$

5.3 Bayesian Update

After each response:

$$P_{t+1}(L) = \frac{P(\text{answer}_t \mid L)P_t(L)}{\sum_k P(\text{answer}_t \mid k)P_t(k)}$$

The posterior becomes the prior for subsequent questions.

5.4 Decision Rule

The system evaluates:

$$P(L \geq \text{target})$$

and applies:

Probability	Verdict	Action
≥ 0.70	Accept	Commit mastery
≤ 0.30	Refresh	Continue learning
$0.30 < P < 0.70$	Ask-more	Collect additional evidence

Table 4: Mastery decision thresholds

The ambiguous region prevents the system from making irreversible decisions from insufficient evidence.

6 Language Model Architecture

Pathfinder supports five interchangeable providers.

Provider	Environment Variable
Anthropic	ANTHROPIC_API_KEY
OpenAI	OPENAI_API_KEY
Google Gemini	GOOGLE_API_KEY
Groq	GROQ_API_KEY
Mistral	MISTRAL_API_KEY

Table 5: Supported model providers

Provider selection is controlled through:

PATHFINDER_PROVIDERS

Unavailable providers are skipped. Provider failure causes failover to the next configured provider. An exhausted spending budget stops failover.

6.1 Structured Output

Model output is constrained to known schemas and revalidated after generation. For example, goal extraction can only return one of the known goal identifiers. A model therefore cannot create a new career or curriculum item.

7 AI Safety and Guardrails

The guardrail architecture is:

Input → Normalization → PII Detection → Injection Detection → Model → Output Validation

7.1 Input Protection

The guard layer performs:

- Unicode NFKC normalization.
- Invisible-character removal.
- PII detection and redaction.
- Luhn-validated card detection.
- Phone and email detection.
- Prompt-injection detection.
- Random per-call input fencing.

Injection categories include:

- Instruction override.

- Prompt disclosure.
- Role reassignment.
- Forged turns.
- Forged fences.
- Jailbreak terminology.
- Exfiltration.
- False authority.

7.2 Output Protection

Generated prose is checked against deterministic application facts.

Validation includes:

- Numbers.
- Catalogue entities.
- Goals and skills.
- Links.
- Forbidden claims.
- Prompt leakage.
- Length and formatting.

Invalid model output is discarded and replaced with deterministic text.

Thus:

Hallucination prevention is enforced by architecture.

8 User Experience

8.1 Offline-First Interaction

Profile changes are immediately reflected using the browser-side engine.

Server responses are debounced by approximately:

200 ms

and replace the local result when available.

8.2 Concurrent Requests

A monotonic request token ensures that an older response cannot overwrite newer state.

8.3 Onboarding

Onboarding presents one question at a time.

The final screen displays the generated learning path before the result is committed.

The same `buildPath()` implementation is used for preview and actual planning.

8.4 Discover

The Discover interface provides searchable resources.

Resource interactions contribute to the same interest and avoidance signals used during onboarding.

8.5 Connection State

The interface distinguishes:

- Standalone.
- API.
- API + model.

This makes dependency availability visible rather than silently hiding degraded operation.

9 API

Endpoint	Purpose	Property
GET /api/health	Health status	Dependency visibility
POST /api/path	Generate path	Deterministic
GET /api/quiz/:skillId	Get assessment	Answer key hidden
POST /api/narrate	Explain path	Output validated
POST /api/onboarding/followup	Follow-up	Closed options
POST /api/onboarding/summary	Summary	Fallback supported

Table 6: Representative API endpoints

All request boundaries are validated.

Malformed requests return HTTP 400 rather than being incorrectly treated as server failures.

10 Performance and Code Quality

Metric	Value
JavaScript bundle	366.26 kB / 113.98 kB gzipped
CSS bundle	35.04 kB / 6.86 kB gzipped
HTML	0.74 kB / 0.45 kB gzipped
Production build	~27 seconds cold
Path generation	Sub-millisecond in browser
Maximum path length	12 items
Model timeout	8 seconds

Table 7: Performance characteristics

The recommendation ordering operates on a bounded set of at most 12 selected items and is therefore computationally small.

Derived Zustand state is memoized to prevent unnecessary rendering.

Strict TypeScript checking is enabled, including unused-variable and unused-parameter checks.

11 Testing

Pathfinder includes 29 end-to-end checks.

The tests verify behavioural properties rather than only HTTP status codes.

Examples include:

- Identical inputs produce identical paths.
- Quiz answers are never exposed.
- Quiz selection is reproducible.
- Correct and incorrect answers move the posterior in opposite directions.
- Unknown goals cannot be generated by the model.
- Invalid onboarding identifiers are rejected.
- Narration cannot reference resources outside the path.
- Malformed requests return HTTP 400.
- Deterministic fallbacks work without a model.

The core tests pass with or without an external model provider.

12 Security

The application applies security controls at multiple layers.

- Strict input validation.
- Helmet security headers.
- Content Security Policy.
- Explicit CORS configuration.
- Request-body size limits.
- Constant-time API-key comparison.
- HTTP-only session cookies.
- Row-Level Security.
- PII redaction before model processing.

Secrets are supplied through environment configuration and are not stored in source code.

13 Deployment

13.1 Local Development

Requirements:

```
Node.js >= 20.19  
npm >= 10
```

Start the frontend:

```
npm install  
npm run dev
```

The frontend runs on:

```
http://localhost:5173
```

Start the backend:

```
npm run server
```

The API runs on:

```
http://127.0.0.1:8787/api
```

13.2 Production

The frontend is built using:

```
npm run build
```

The resulting Vite application is placed in **dist/**.

The backend can be deployed as an Express serverless function.

The production architecture is therefore:

Browser → Static Frontend

and

Browser → Serverless Express API.

14 Failure Handling

Pathfinder follows graceful degradation.

Failure	Response
Model unavailable	Deterministic fallback
Model timeout	Abort after 8 seconds
Invalid model output	Reject and fallback
Provider failure	Try next provider
Prompt injection	Reject or sanitize
Malformed request	HTTP 400
Stale response	Ignore older result
Network unavailable	Continue locally

Table 8: Failure-handling strategy

The core principle is:

AI failure $\neq$ application failure

15 Innovation

Pathfinder introduces several architectural ideas beyond a standard AI curriculum recommender.

15.1 Model as Renderer

The model does not create the learning plan.

Rules Engine $\rightarrow$ Plan $\rightarrow$ LLM $\rightarrow$ Explanation

This makes recommendations auditable.

15.2 Evidence-Based Explanations

Explanations are derived from the same calculations that generated the recommendation.

Therefore, the explanation can reference actual:

- Skill movement.
- Required skills.
- Selected resources.
- Interest matches.

15.3 Probabilistic Skipping

Mastery decisions use posterior probability instead of a binary “passed” flag.

This reduces the risk of incorrectly skipping required material.

15.4 Honest Degradation

The system exposes whether it is operating standalone, with the API, or with an active model.

The user can therefore distinguish normal operation from degraded operation.

16 Limitations and Future Work

16.1 Current Limitations

The current implementation has a deliberately bounded catalogue:

- 27 skills.
- 35 learning resources.
- 4 career tracks.

The mastery model uses six latent levels and a simple response likelihood rather than a full Item Response Theory model.

Career information is also limited by the available catalogue.

16.2 Future Improvements

Potential extensions include:

- Larger skill ontology.
- Full IRT-based assessment modelling.
- Continuous labour-market data updates.
- Learning-velocity modelling.
- More detailed personalization.
- Additional model providers.
- Expanded career coverage.

17 Conclusion

Pathfinder demonstrates an architecture in which AI enhances a career-development platform without becoming its source of truth.

The deterministic recommendation engine handles:

- Skill-gap analysis.
- Resource selection.
- Dependency ordering.
- Learning-path generation.

The Bayesian layer handles uncertainty in learner mastery.

The language-model layer handles:

- Natural-language interpretation.
- Explanations.

- Conversational responses.

Guardrails, output validation, provider failover, and deterministic fallbacks ensure that external AI failures do not compromise the core application.

The resulting architecture is:

Deterministic Planning + Probabilistic Assessment + Constrained AI = Reliable Career Intelligence
